SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Mock Interview (Low)
Assessment Tool Weightage: 30%

Dr

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	**5**
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	**5**
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	**5**
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	**5**
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	**5**
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	**5**
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	**5**
8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	**5**
9	Interviewer: 'What is a serializability schedule? How do you test for conflict	BL4 – Analyze	**5**

	serializability using a precedence graph?'		
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	**5**

Code of Conduct:

I M.Devi Srilatha(192525236) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student
Srilatha

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively

Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

1. Interviewer: ‘Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?’

Query processing in a DBMS is the process of converting an SQL query into an efficient execution plan and then executing it to produce the result.

Steps of query processing:

1. **Parsing and Translation**
    - The DBMS checks the SQL query for syntax and semantic errors and converts it into an internal representation, such as a relational algebra expression.
2. **Query Optimization**
    - The optimizer generates different possible execution plans.
    - It estimates the cost of each plan based on factors such as:
        - Number of rows
        - Available indexes
        - Join methods
    - It selects the plan with the lowest estimated cost.
3. **Query Execution**
    - The selected execution plan is passed to the execution engine.
    - Operations such as selection, projection, sorting, and joins are performed.
4. **Result Generation**
    - The execution engine produces the final result and returns it to the user.

How does the optimizer choose the best plan?

It compares their estimated costs using table size, indexes, available memory, and expected disk I/O. The optimizer chooses the plan that is expected to execute the query most efficiently.

Simple flow:

SQL Query → Parser → Query Tree → Query Optimizer → Best Execution Plan → Execution Engine → Result

Interview one-line answer:
“The main goal of query optimization is to produce the same result with minimum execution cost and better performance.”

2. Interviewer: What is the difference between heuristic optimization and cost-based optimization? When would you use each?

The main difference is **how the optimizer chooses the execution plan**.

Heuristic Optimization	Cost-Based Optimization
Uses predefined rules to improve the query.	Calculates the estimated cost of different plans.
Does not require detailed database statistics.	Uses statistics such as table size, number of rows, and indexes.
Faster and simpler.	More complex but usually produces better plans.
Example: Apply selection before joining tables.	Example: Choose between hash join, nested-loop join, or sort-merge join based on cost.
May not always choose the optimal plan.	Usually chooses the plan with the lowest estimated cost.

When to use each?

Heuristic optimization:
Use it when a quick and simple optimization is needed, especially for straightforward queries where standard rules can improve performance.

Cost-based optimization:
Use it for complex queries involving multiple joins, large tables, indexes, and different possible execution plans. It is preferred when performance is important.

Heuristic = Rule-based

Cost-based = Estimate cost and choose the cheapest plan

Interview one-line answer:
Heuristic optimization uses simple rules to choose a query plan, while cost-based optimization compares the cost of different plans and chooses the cheapest one.

3. Interviewer: How would you implement database connectivity in areal project? Explain the difference between JDBC and ODBC.

In a real project, I would implement database connectivity by using a **database driver and connection manager**.

Steps to implement database connectivity:

1. **Choose the database** – For example, MySQL.
2. **Add the database driver** – In Java, add the MySQL JDBC driver.
3. **Create a connection** – Use the database URL, username, and password.
4. **Execute queries** – Use PreparedStatement for parameterized queries.
5. **Process results** – Use ResultSet to read data returned by SELECT queries.
6. **Handle exceptions** – Use exception handling and rollback transactions when required.
7. **Close resources** – Close the connection, statement, and result set properly.
8. **Use connection pooling** – In large applications, reuse database connections instead of creating a new connection for every request.

JDBC vs ODBC

JDBC	ODBC
Stands for Java Database Connectivity.	Stands for Open Database Connectivity.
Mainly used by Java applications.	Can be used by applications written in different programming languages.
Uses JDBC drivers.	Uses ODBC drivers.
Designed specifically for Java.	Language-independent database connectivity standard.
Commonly used in Java web and enterprise applications.	Commonly used for connecting various applications to databases through ODBC-compatible drivers.

Interview one-line answer:

JDBC is used to connect Java applications to databases, while ODBC is a general standard for connecting different applications to databases. In a real project, I would use JDBC with a database driver to connect and run SQL queries safely.

4. Interviewer: What are ACID properties? Give a real-world example where each property is critical.

ACID properties ensure that database transactions are processed **reliably and correctly**. ACID stands for **Atomicity, Consistency, Isolation, and Durability**.

Property	Meaning	Real-world example
Atomicity	A transaction is completed fully or not at all.	In a bank transfer, ₹1,000 must be deducted from one account and added to another. If one operation fails, the entire transaction is rolled back.
Consistency	The database must remain in a valid state before and after a transaction.	When transferring money, the account balance cannot become invalid or violate database rules.
Isolation	Concurrent transactions should not interfere with each other.	If two people try to book the last available seat simultaneously, only one booking should succeed.
Durability	Once a transaction is committed, the changes are permanently stored.	After a successful online payment, the payment record should remain saved even if the server crashes immediately afterward.

Simple Example

Consider an **online banking money transfer**:

Account A → ₹1,000 → Account B

- **Atomicity:** Both debit and credit happen, or neither happens.
- **Consistency:** The balances remain correct.
- **Isolation:** Other transactions cannot see incomplete transfer results.
- **Durability:** After confirmation, the transfer remains recorded even after a system failure.

Interview one-line answer:

"ACID properties make transactions reliable: Atomicity ensures all-or-nothing execution, Consistency maintains valid data, Isolation prevents interference between concurrent transactions, and Durability ensures committed data is not lost."

5. Interviewer: Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?

Concurrency control manages multiple transactions running at the same time so that the database remains correct and consistent.

1. Dirty Read

A **dirty read** occurs when one transaction reads data that another transaction has modified but **not yet committed**.

Example:

- T1 changes balance from ₹5,000 to ₹3,000.
- T2 reads ₹3,000.
- T1 rolls back.
- The actual balance becomes ₹5,000, but T2 had already read the incorrect value.

2. Lost Update

A **lost update** occurs when two transactions update the same data, and one update **overwrites the other**.

Example:

- T1 reads stock = 10.
- T2 also reads stock = 10.
- T1 updates stock to 8.
- T2 updates stock to 7.
- T1's update is lost.

3. Unrepeatable Read

An **unrepeatable read** occurs when a transaction reads the same row twice and gets **different values** because another transaction modified and committed the row between the two reads.

Example:

- T1 reads salary = ₹30,000.

- T2 changes salary to ₹35,000 and commits.
- T1 reads the salary again and gets ₹35,000.

How does 2PL solve these problems?

2PL (Two-Phase Locking) controls access to data using locks.

It has two phases:

1. **Growing Phase** – The transaction can acquire locks but cannot release them.
2. **Shrinking Phase** – The transaction can release locks but cannot acquire new locks.

Types of locks:

- **Shared lock (S):** Used for reading.
- **Exclusive lock (X):** Used for writing.

For example, if T1 has an **exclusive lock** on a row, T2 cannot read or modify that row until T1 releases the lock. This prevents conflicting operations.

Therefore, 2PL helps prevent **lost updates, dirty reads, and unrepeatable reads** and produces **conflict-serializable schedules**.

Interview one-line answer:

"Concurrency problems occur when transactions access the same data simultaneously. Two-Phase Locking uses shared and exclusive locks to control these accesses, ensuring conflicting transactions wait for each other and maintaining database consistency."

6. Interviewer: What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.

A **deadlock** in DBMS occurs when two or more transactions are waiting for each other to release resources, so none of them can continue.

Example

- **T1** locks table/row A and waits for B.
- **T2** locks table/row B and waits for A.
- T1 waits for T2, and T2 waits for T1.
- Both transactions are stuck.

Deadlock Detection Algorithm

The DBMS can use a **Wait-For Graph** to detect deadlocks.

1. Create a node for each active transaction.
2. Draw an edge **T1 → T2** if T1 is waiting for a resource held by T2.
3. Check the graph for a **cycle**.
4. If a cycle exists, a deadlock has occurred.
5. Select one transaction as the **victim**.
6. Roll back the victim transaction and release its locks.
7. Allow the remaining transaction to continue.

Example:

T1 → T2 → T1

This cycle indicates a deadlock.

Deadlock Prevention Strategies

1. Wait-Die

- Assign each transaction a timestamp.
- If an older transaction requests a lock held by a younger transaction, the older transaction waits.
- If a younger transaction requests a lock held by an older transaction, the younger transaction is rolled back.

2. Wound-Wait

- If an older transaction requests a lock held by a younger transaction, the older transaction forces the younger transaction to roll back.
- If a younger transaction requests a lock held by an older transaction, the younger transaction waits.

Another simple strategy is **resource ordering**, where all transactions request resources in the same predefined order. This prevents circular waiting.

Interview one-line answer:

"Deadlock occurs when transactions wait for each other indefinitely. DBMS detects it using a wait-for graph and identifies a cycle. It can prevent deadlocks using techniques such as wait-die, wound-wait, or resource ordering."

7. Interviewer: Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?

Immediate Update and **Deferred Update** are two DBMS recovery techniques that determine when changes made by a transaction are written to the database.

Immediate Update	Deferred Update
Changes can be written to the database before the transaction commits.	Changes are not written to the database until the transaction commits.
Uses **UNDO and REDO** during recovery.	Mainly requires **REDO** during recovery.
Uncommitted changes may exist on disk.	Uncommitted changes are not written to the database.
More flexible for buffer management.	Simpler recovery mechanism.
Recovery is more complex.	Recovery is comparatively easier.

Immediate Update

In immediate update, database changes can be written before COMMIT.

Example:

- T1 changes balance from ₹5,000 to ₹4,000.
- The change is written to disk.
- System crashes before T1 commits.
- DBMS uses **UNDO** to restore ₹5,000.

Preferredwhen:
The system needs efficient buffer management and high-performance transaction processing where data pages may be written before commit.

Deferred Update

In deferred update, changes are kept in memory/log until the transaction successfully commits. Only committed changes are applied to the database.

Example:

- T1 changes balance from ₹5,000 to ₹4,000.
- System crashes before T1 commits.
- Since the change was not written to the database, no UNDO is required.
- If T1 had committed before the crash, **REDO** can apply the change.

Preferredwhen:
Simple and reliable recovery is more important and the system can delay writing database changes until commit.

Easy way to remember

Immediate Update → UNDO + REDO
Deferred Update → REDO

Interview one-line answer:

"Immediate update allows changes to reach the database before commit, so recovery may need both UNDO and REDO. Deferred update writes changes only after commit, so recovery is simpler and mainly uses REDO. Immediate update is preferred for flexibility and performance, while deferred update is preferred when simpler recovery is desired."

8. Interviewer: Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?

Shadow Paging is a DBMS recovery technique that maintains **two page tables**:

- **Shadow Page Table** – points to the original database pages.
- **Current Page Table** – points to pages being modified by the transaction.

How Shadow Paging Works

1. Initially, the **shadow page table** and current page table point to the same database pages.
2. When a transaction modifies a page, the original page is **not overwritten**.
3. A new page is created, and the updated data is written to it.
4. The current page table is updated to point to the new page.
5. If the transaction commits, the current page table becomes the new valid page table.
6. If the system crashes before commit, the DBMS discards the current page table and uses the **shadow page table** to restore the previous state.

Simple flow:

Original Pages → Shadow Page Table

Updated Pages → Current Page Table

COMMIT → Current Table becomes valid

CRASH → Discard Current Table → Use Shadow Table

Advantages Compared with Log-Based Recovery

1. **No complex UNDO/REDO processing** is normally required.
2. **Fast recovery** because the DBMS can use the shadow page table directly.
3. The original database pages remain safe until commit.
4. It is relatively simple to understand and implement for suitable workloads.

Limitations

1. **Extra storage** is required because modified pages are copied.
2. Page-table management can create significant overhead.
3. **Fragmentation** can occur because updated pages may be stored in different locations.
4. It is less suitable for **large databases and high-concurrency systems**.

5. Log-based recovery is generally more flexible and scalable for modern DBMSs.

Interview one-line answer

“Shadow paging maintains a shadow copy of the page table and writes updates to new pages. It provides fast and simple recovery without traditional UNDO/REDO processing, but it requires extra storage, causes page-management overhead and fragmentation, making log-based recovery more suitable for large, high-concurrency databases.”

9. Interviewer: What is a serializability schedule? How do you test for conflict serializability using a precedence graph?

A **schedule** is the order in which operations of multiple transactions are executed. A schedule is **serializable** if its final result is equivalent to some serial execution of those transactions.

Conflict serializability means that a schedule can be transformed into a serial schedule by swapping operations that do not conflict.

Testing Conflict Serializability Using a Precedence Graph

We use a **precedence graph** (also called a serialization graph).

Steps:

1. Create one **node for each transaction**.
2. Check the schedule for conflicting operations.
3. Two operations conflict when:
    - They belong to different transactions.
    - They access the same data item.
    - At least one operation is a **write**.
4. If transaction **T1** performs the conflicting operation before **T2**, draw an edge: **T1 → T2**
5. Check the graph for a **cycle**.

Decision

- **No cycle → Conflict serializable**
- **Cycle exists → Not conflict serializable**

Example

Suppose the schedule contains:

T1:R(A) → W(A)
T2: R(A) → W(A)

If T1's write on A occurs before T2's read/write on A, we get:

T1 → T2

Since there is no cycle, the schedule is **conflict serializable**, equivalent to the serial order **T1 → T2**.

If the graph contains:

T1 → T2 → T3 → T1

there is a cycle, so the schedule is **not conflict serializable**.

Interview one-line answer:

"A schedule is conflict serializable if it is equivalent to a serial schedule based on conflicting operations. We create a precedence graph with transactions as nodes and edges representing conflicts. If the graph has no cycle, the schedule is conflict serializable; if it has a cycle, it is not."

10. Interviewer: Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?

Two-Phase Locking (2PL) is a concurrency-control protocol that ensures **conflict serializability** by controlling when transactions acquire and release locks.

Types of 2PL

Type	Description	Main Feature
Basic 2PL	A transaction has a growing phase where it acquires locks and a shrinking phase where it releases locks.	Guarantees conflict serializability.
Strict 2PL	All **exclusive (X) locks** are held until the transaction commits or aborts.	Prevents dirty writes and simplifies recovery.
Rigorous 2PL	Both **shared (S) and exclusive (X) locks** are held until commit or abort.	Provides stronger isolation and a clear serial order.

1. Basic 2PL

It has two phases:

- **Growing phase:** The transaction can acquire locks but cannot release them.
- **Shrinking phase:** The transaction can release locks but cannot acquire new locks.

It guarantees **conflict serializability**, but early lock releases can cause cascading rollbacks.

2. Strict 2PL

In Strict 2PL, **exclusive locks are held until commit or abort**.

This prevents other transactions from reading or overwriting data modified by an uncommitted transaction.

Advantage: Recovery becomes easier because cascading rollbacks are avoided.

3. Rigorous 2PL

Rigorous 2PL holds **all locks, both shared and exclusive, until the transaction finishes**.

It provides stronger control than Strict 2PL and makes the serialization order easier to determine.

Which is most commonly used?

Strict 2PL is the most commonly used variation in traditional DBMS systems because it provides a good balance between **concurrency, consistency, and easy recovery**.

Interview one-line answer:

"Basic 2PL guarantees conflict serializability, Strict 2PL holds exclusive locks until commit or abort, and Rigorous 2PL holds both shared and exclusive locks until commit or abort. Strict 2PL is commonly preferred because it prevents cascading problems and simplifies recovery while still allowing reasonable concurrency."